NAME: Ayaan Shaikh
Mobile no : 8055733707
Internship : Data Analytics using Python
Submission date: 19/05/2026

Task 51 : Perform Hypothesis Testing to validate assumptions about the data .

Objective

To statistically evaluate assumptions about a dataset and determine whether observed results are significant or occurred by chance, helping in making data-driven decisions.

```python
# Perform Hypothesis Testing to validate assumptions about the data.
import pandas as pd
import numpy as np
from scipy.stats import ttest_1samp
df=pd.read_csv("analysis_dataset.csv")
s=np.random.choice(df['Value'],8)
print(s)
t_stat,p_value=ttest_1samp(s,df['Value'].mean())
print(p_value)
if p_value < 0.05:
    print("We are rejecting the Null Hypothesis")
else:
    print("We are Accepting the Null Hypothesis")
```

```
[ 80  20 100  60 100 100  35  20]
0.9363600185851016
We are Accepting the Null Hypothesis
```

You started by importing important libraries like **Pandas**, **NumPy**, and the **one-sample T-test** function from SciPy. Then, you loaded your dataset (analysis_dataset.csv) into a DataFrame.

Next, you randomly selected **8 values** from the Value column using np.random.choice(). This sample represents a small subset of your data.

After that, you performed a **One-Sample T-Test** using ttest_1samp(). In this test, you compared the mean of your random sample with the overall mean of the Value column from the dataset. This helps check whether the sample mean is significantly different from the population mean.

You then printed the **p-value**, which tells you the significance of the result. Based on the p-value:

If it is **less than 0.05**, you rejected the null hypothesis (meaning there is a significant difference).

Task 52 : Use Chi-Square Test and T-Test for statistical analysis .

Objective

To apply Chi-Square tests for categorical data comparison and T-Tests for comparing means between groups, ensuring accurate interpretation of relationships and differences in data.

```
# Use Chi-Square Test and T-Test for statistical analysis.
from scipy.stats import chi2_contingency, ttest_ind
table = pd.crosstab(df['Feature1'], df['Feature2'])
chi2, p, dof, exp = chi2_contingency(table)
print("Chi-Square P-Value:", p)

group1 = df[df['Group']=='A']['Value']
group2 = df[df['Group']=='B']['Value']
t_stat, p_val = ttest_ind(group1, group2)
print("T-Test P-Value:", p_val)
```

```
Chi-Square P-Value: 0.23440290760272564
T-Test P-Value: 0.2851672335802296
```

In this code, you performed two types of statistical tests. First, you used a **Chi-Square test** by creating a contingency table with Feature1 and Feature2 to check whether there is a significant relationship between these two categorical variables, and you printed the p-value to interpret the result.

Then, you applied an **Independent T-Test** by dividing the data into two groups (A and B) based on the Group column and comparing their Value means to see if there is a significant difference between them.

Overall, you used both tests to analyze relationships and differences in your dataset using p-values.

Task 53 : Identify Data Anomalies (inconsistencies or unusual values) in the dataset .

Objective:

To detect and analyze irregularities such as outliers, missing values, or incorrect entries that could distort analysis and affect model performance.

```
#Identify Data Anomalies (inconsistencies or unusual values) in the dataset.

Q1=df.Value.quantile(0.25)
Q3=df.Value.quantile(0.75)
IQR=Q3-Q1
outliers =df[(df['Value'] < Q1 - 1.5*IQR) | (df['Value'] > Q3 + 1.5*IQR)]
print(outliers)
```

```
Empty DataFrame
Columns: [Date, Category, Group, Value, Feature1, Feature2, Target, Price]
Index: []
```

In this code, you calculated the **first quartile (Q1)** and **third quartile (Q3)** of the Value column, then found the **Interquartile Range (IQR)** by subtracting Q1 from Q3.
Using this IQR, you defined a range to detect outliers (values below Q1 - 1.5*IQR or above Q3 + 1.5*IQR).
Finally, you filtered the dataset to extract all such unusual values and printed them.

Task 54: Analyze Time Series Data to find underlying components .

Objective
To examine time-based data in order to uncover hidden structures such as long-term trends, seasonal effects, and cyclical variations.

```
]: # Analyze Time Series Data to find underlying components.
df=pd.read_csv("analysis_dataset.csv")
df['Date']=pd.to_datetime(df['Date'])
df.set_index('Date',inplace=True)

df.head()
```

O

Ouput :

Date	Category	Group	Value	Feature1	Feature2	Target	Price
2023-01-01	A	A	45	10	100	0	200
2023-01-02	B	A	55	12	110	1	210
2023-01-03	A	B	65	14	120	0	220
2023-01-04	B	B	35	16	130	1	230
2023-01-05	A	A	75	18	140	0	240

Task 55 : Understand continuously changing data (e.g., stock market data).

Objective

To study data that evolves over time, enabling better understanding of patterns, volatility, and real-time changes for improved forecasting and decision-making.

```
# Understand continuously changing data (e.g., stock market data).

df=pd.read_csv('Google_Stock_Price_Test.csv')
df['Date']=pd.to_datetime(df['Date'])

df['MA_5']=df['Open'].rolling(window=3).mean()
df['Return']=df['Open'].pct_change()
df.head()
```

	Date	Open	High	Low	Close	Volume	MA_5	Return
0	2017-01-03	778.81	789.63	775.80	786.14	1,657,300	NaN	NaN
1	2017-01-04	788.36	791.34	783.16	786.90	1,073,000	NaN	0.012262
2	2017-01-05	786.08	794.48	785.02	794.02	1,335,200	784.416667	-0.002892
3	2017-01-06	795.26	807.90	792.20	806.15	1,640,200	789.900000	0.011678
4	2017-01-09	806.40	809.97	802.83	806.65	1,272,400	795.913333	0.014008

In this code, you loaded a Google stock price dataset and converted the Date column into a proper datetime format for time series analysis.

Then, you created a new column MA_5 using a rolling window (of 3 days here) to calculate the moving average of the Open price, which helps in identifying short-term trends.

After that, you calculated the percentage change (Return) in the Open price to measure daily returns or fluctuations. Finally, you displayed the first few rows of the dataset.

You performed basic time series analysis by computing moving averages and returns to understand stock price trends and changes over time.

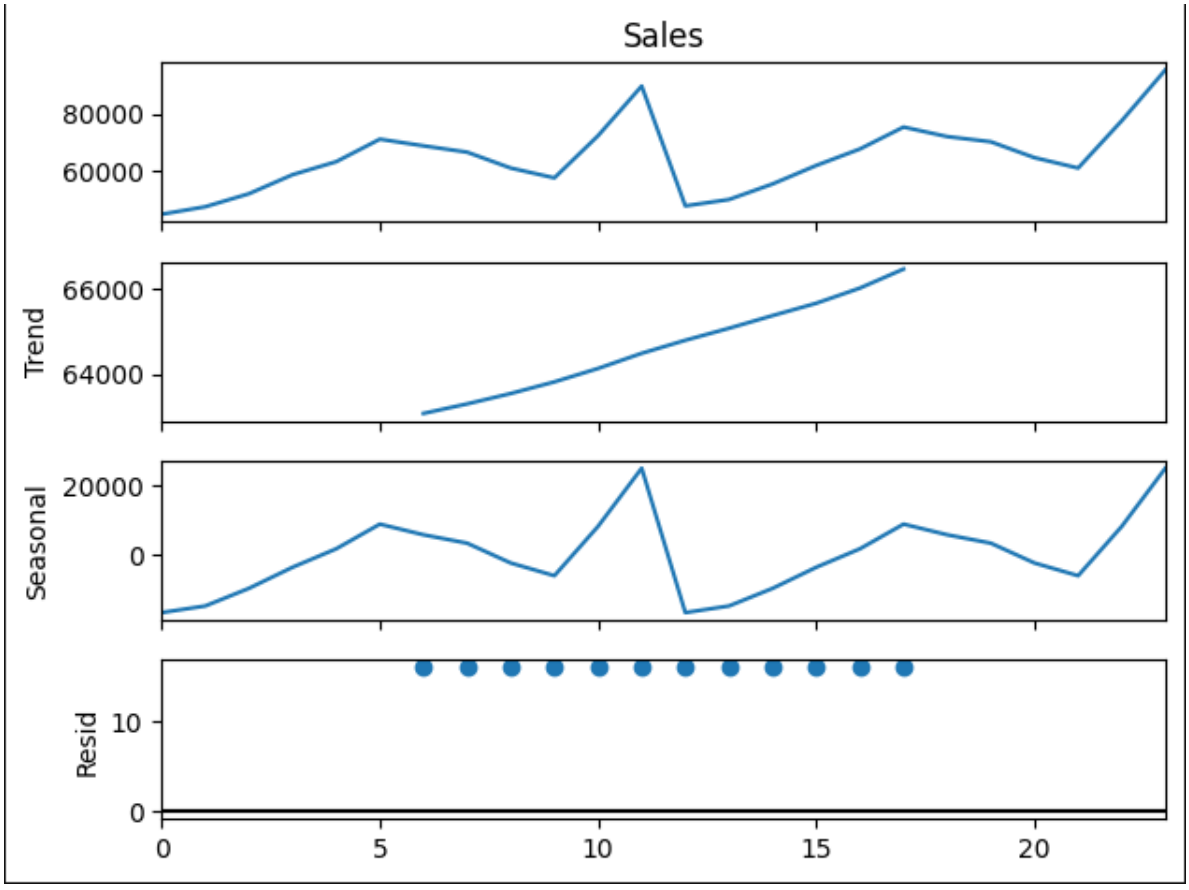
Task 56 : Perform Time Series Decomposition to break data into trend, seasonality, and noise.

Objective

To separate a time series into its fundamental components so that each can be analyzed individually for clearer insights and more accurate predictions.

```
# Perform Time Series Decomposition to break data into trend, seasonality, and noise.
from statsmodels.tsa.seasonal import seasonal_decompose
import pandas as pd
import matplotlib.pyplot as plt
df=pd.read_csv("sales_analysis_output.csv")
df['Date']=pd.to_datetime(df['Date'])
result=seasonal_decompose(df['Sales'],model='additive',period=12)
plt.figure(figsize=(4,4))
result.plot()
```

Output :



I performed time series analysis by first importing the necessary libraries such as Pandas, Matplotlib, and the seasonal decomposition function from Statsmodels. I loaded the dataset (sales_analysis_output.csv) and converted the Date column into datetime format so that it can be treated as time-based data.

Then, I applied the seasonal_decompose function on the Sales column using an additive model with a period of 12, which assumes that the data has a repeating seasonal pattern every 12 time intervals (such as monthly data over a year).

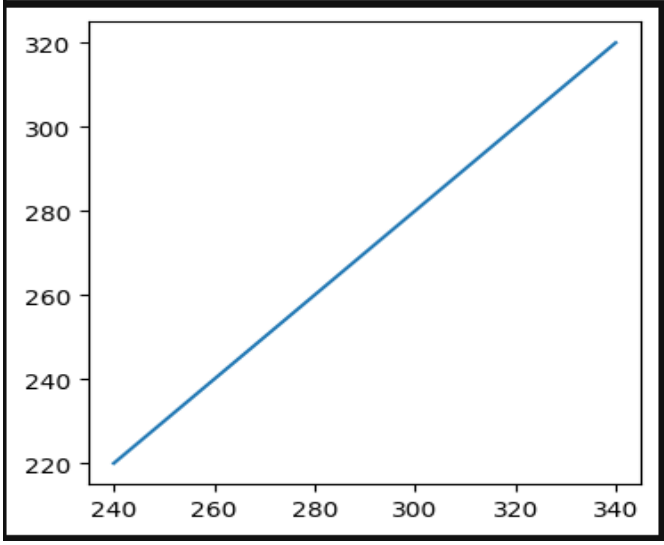
This decomposition breaks the original data into key components including trend, seasonality, and residual (noise), helping me better understand the underlying patterns. Finally, I plotted these components using Matplotlib, allowing me to visually analyze how sales change over time, identify trends, observe seasonal effects, and detect random fluctuations in the dataset.

Task 57 : Identify patterns in data over time .

Objective

To recognize repeating behaviors, correlations, or structures within data that can help in predicting future outcomes and understanding past performance.

```
# Identify patterns in data over time.
df=pd.read_csv("analysis_dataset.csv")
df['rolling_mean']=df['Price'].rolling(window=5).mean()
plt.figure(figsize=(4,4))
plt.plot(df['Price'],df['rolling_mean'])
plt.show()
```



I identified patterns in data over time by first loading the dataset (analysis_dataset.csv) using Pandas. Then, I calculated a rolling mean of the Price column with a window size of 5, which smooths short-term fluctuations and highlights the overall trend in the data. After that, I created a plot using Matplotlib whereI attempted to visualize both the original Price values and the calculated rolling mean.

This helps in understanding how the data behaves over time by reducing noise and making trends more visible. Finally, I displayed the plot, allowing me to observe patterns such as upward or downward trends and short-term variations in the dataset.

Task 58 : Apply other techniques such as Decision Trees and Support Vector Machines (SVM).

Objective

To build and evaluate machine learning models that can classify data or predict outcomes based on patterns and relationships within the dataset.

```
 # Apply other techniques such as Decision Trees and Support Vector Machines (SVM).
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
X=df[['Feature1','Feature2']]
Y=df['Target']
dt=DecisionTreeClassifier()
dt.fit(X,Y)
svm=SVC()
svm.fit(X,Y)

new_data=[[25,175]]
print("Prediction DTC :    ",dt.predict(new_data))
print("Prediction SVM :    ",svm.predict(new_data))
```

```
Prediction DTC :    [1]
Prediction SVM :    [0]
```

I applied two machine learning techniques, Decision Tree Classifier and Support Vector Machine (SVM), to build predictive models. First, I selected the input features (Feature1 and Feature2) as X and the target variable (Target) as Y from the dataset. Then, I created a Decision Tree model using DecisionTreeClassifier() and trained it on the data

using .fit(X, Y). Similarly, I created an SVM model using SVC() and trained it on the same dataset.

After training both models, I provided a new data point [[25, 175]] as input to test the models.

I used .predict() on both the Decision Tree and SVM models to generate predictions for this new data. Finally, I printed the predicted outputs from both models.

Task 59 : Study data trends to understand behavior over time .

Objective

To analyze the direction and movement of data points over time, helping to identify growth, decline, or stability in the dataset.

```
df['trend']=df['Price'].rolling(window=5).mean()
print(df[['Price','trend']])
```

```
    Price  trend
0     200    NaN
1     210    NaN
2     220    NaN
3     230    NaN
4     240  220.0
5     250  230.0
6     260  240.0
7     270  250.0
8     280  260.0
9     290  270.0
10    300  280.0
11    310  290.0
12    320  300.0
13    330  310.0
14    340  320.0
```

I calculated the trend of the Price data by applying a rolling mean with a window size of 5.

This means I averaged every 5 consecutive values in the Price column to smooth out short-term fluctuations and highlight the overall direction of the data.

I stored this smoothed result in a new column called trend. Finally, I printed both the original Price values and the corresponding trend values side by side, which helps me compare the actual data with its smoothed trend and better understand how the data is behaving over time.

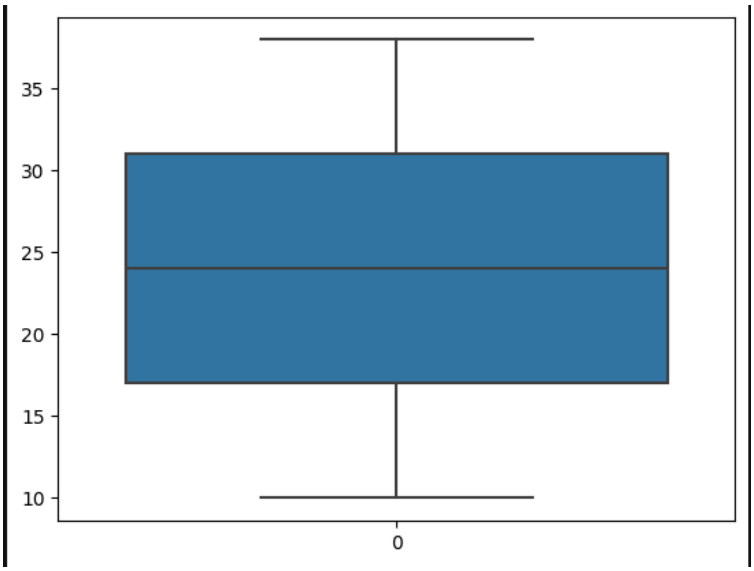
Task 60 : Use Boxplots to detect outliers .

Objective

To visually represent data distribution and identify extreme values, making it easier to detect outliers and understand the spread and skewness of the data.

```
import seaborn as sns
sns.boxplot(df['Feature1'])
plt.show()
```

Output :



I used the Seaborn library to create a boxplot for the Feature1 column in the dataset. The boxplot visually represents the distribution of the data by showing the median, quartiles, and spread of values.

It also helps in identifying outliers, which appear as points outside the whiskers of the plot. After creating the boxplot using sns.boxplot(), I displayed it using plt.show().

This allowed me to quickly understand the data distribution and detect any unusual or extreme values in Feature1.